

Let's Learn React

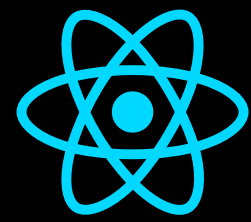
Presented by: Aniyah, Ashley, Imani, Jael, & Wynter

What's React?

React Background

A JavaScript library for building interactive user interfaces!

- Component-based Architecture: Create reusable components with states and behaviors
- Virtual Document Object Model (DOM): Updates UI in response to state changes; dynamic applications

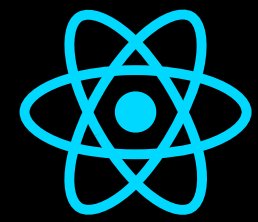


React Framework

- **Type:** Multi-paradigm (supports object-oriented & functional styles)
- **Key Features:** Component-based architecture, virtual DOM, state management
- **Strengths:** Responsive UI, reusable components, large ecosystem
- **Limitations:** Requires setup environment, reliance on JavaScript

Special Features

- JSX Syntax: Combines JavaScript and HTML-like syntax
- State Management: Uses hooks like *useState* and *useEffect* for managing state and side effects
- Component Reusability: Promotes reusable, isolated components that manage their own logic



React Benefits

- React's ecosystem includes numerous libraries (Redux), making it *suitable for complex applications*
- Offers excellent support for building *responsive, interactive web interfaces*

The background is a dark blue-grey color filled with various light grey, hand-drawn sketches. These sketches include atomic models with orbits, light bulbs, arrows, geometric shapes like triangles and squares, and some faint text like 'S', 'N', and 'A'.

**ARE YOU SMARTER
THAN A 5TH GRADER**

Code Snippet (Scheme)

```
(define (validate-answer user-answer correct-answer)
  (if (equal? user-answer correct-answer)
      'correct
      'incorrect))
(display (validate-answer "Paris" "Paris")); Output: correct
```


Code Snippet

```
import React, { useState } from 'react';
function AnswerValidator({ userAnswer, correctAnswer }){
  const [result, setResult] = useState('');
  const validateAnswer = () => {
    if (userAnswer.toLowerCase() === correctAnswer.toLowerCase())
    {setResult('Correct!');
    }
    else {
      setResult('Incorrect!');
    }
  };

  return (
    <div>
      <button onClick = {validateAnswer}>Submit Answer</button>
      <p>{result}</p>
    </div>
  );
}
export default AnswerValidator;
```

Code Snippet (React/JavaScript)

```
import React, { useState } from 'react';
function AnswerValidator({ userAnswer, correctAnswer }) {
  const [result, setResult] = useState('');
  const validateAnswer = () => {
    if(userAnswer.toLowerCase() === correctAnswer.toLowerCase())
    {setResult('Correct!');
    }
    else {
      setResult('Incorrect!');
    }
  };
};
```

Code Snippet (React/JavaScript)

```
return (  
  <div>  
    <button onClick = {validateAnswer}>Submit Answer</button>  
    <p>{result}</p>  
  </div>  
);  
}  
export default AnswerValidator;
```

What's the Problem?

The Problem

- User answer questions from various categories
- Game checks answers & provides feedback
- Accumulates score based on correct answers

The Goals

- Core game logic in React & Scheme
- Provide interactive feedback
- Showcase functional programming principles & component-based architecture

Scheme Implementation

- Main Game Loop: Manages game flow & prompts user with questions
- Question Function: Displays question & checks user answer using validate-answer
- Score Tracker: Keep track of user's correct & incorrect answers, displays final score

React Implementation

- App Component: Main entry point, manages game state
- QuestionDisplay Component: Renders current question & answer options
- AnswerValidator Component: Checks user input & updates result
- ScoreDisplay Component: Shows current score
- GameEndScreen Component: Displays final score with congratulatory message

User Flow

- Start Game: Press “Start” button to begin quiz
- Question Display: Multiple-choice options using radio buttons
- Answer Submission: Select answer, click “Submit” to trigger validation
- Feedback Display: Shows “Correct!” or “Incorrect!” based on user input
- Game End: Display final score with option to restart

Fun Facts!

Humble Beginnings



Originally created by **Jordan Walke**, a software engineer at Facebook, to solve the problem of efficiently updating the UI
(Released May 2013)

The Facebook Connection



React is maintained by
Facebook, ensuring its
continued development
and a **strong foundation**

Powering the Web



React is used by countless websites and apps, including **Facebook, Instagram, Netflix,** and many more!

Conclusion

React over C++?

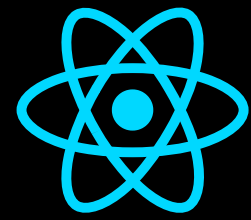
- **UI Development:** In React, able to describe UI state, then framework handles updates automatically. Versus manually manipulating DOM in C/C++
- **Hot Reloading:** React provides real-time changes without full page refreshes, whereas C/C++ requires full recompilation (compiling)
- **Large Ecosystem:** React's rich libraries & tools makes it easier to build applications. C/C++ have limited high-level tools & resources

Pros

- Easy writability with JSX syntax extension
- Build native mobile apps for iOS and Android
- A large community with extensive support & resources

Cons

- Updates & new libraries can be overwhelming
- Mastering React & its ecosystem takes time
- Without state management library, can be complex



React Conclusion

React & Scheme highlight the *diverse approaches* when developing interactive educational applications.

Functional programming + Modern web development

→ Scheme: Concise, logic-focused approach

→ React: Excels in creating dynamic, user-friendly interfaces

Questions?
